Software Design Document (SDD)
Assignment #2 Limited Two-Pass XE Assembler
CS530

Team:
Brian Madott, cssc2537, [tasks includes: design, developer (Opening/Verifying input files, Pass 1, Symbol Table), test, documentation]
Minh Tran, cssc2552, [tasks includes: design, developer (Opcode table, Pass 2, Output symbol table file and a listing file), test, documentation]

Overview & Goals:
Major Overview: For this CS 530 project, our team will be creating a two-pass SIC/XE assembler on C++ language. The program will read XE assembly source files and will return a symbol table file and a listing file.

Sub Goals:
1. Open and read input files(XE source files) as command-line arguments. The program will terminate if no input files are given
2. Implement first pass logic by creating a symbol table and effectively supporting/processing assembler directives (up to 2.3.4 on the textbook)
3. Implement second pass logic by using the generated symbol table (and opcode table) to generate a listing file
4. Print the generated symbol table file and the listing table file
5. Ensure that the program will successfully run at the Edoras Linux server

Project Description:
The purpose of this project is to create a two-pass assembler that handles all types of addressing modes (immediate, indirect, simple, extended format, indexed, PC relative, and Base relative). The program will read necessary input XE source files as command-line arguments and process the source files into two passes. In the first pass of the assembler, the program will assign memory addresses to instructions, build a symbol table (define symbols and corresponding memory address), and process necessary assembly directives. The information generated in the first pass (Symbol table and Opcode table, which opcode table will be created manually) will be used in the second pass phase to produce a listing file and a symbol table file. If there are errors to the input files or if the user does not pass any command-line argument, the program will terminate and will output an IO message to inform the user to pass the correct input files.

Plan of Action and Milestones (POAM) + Estimated hours:
Include each task, milestones (dates) for each task, dependencies/critical path
<u>Phase One: Outline/ Brainstorm (Mar 8th - Mar 10th)</u>
- Discuss system design and requirements, including SIC/XE instruction formats, addressing modes and assembler directives.
- Design overall two-pass architecture and understand what each pass does.
- Identify input/output file formats and handle any possible errors.

- Discuss timeframe and workload distribution.
- Submit Software Design Document on Mar 10th.

Estimated Hours: 4

<u>Phase Two: Design Core functions and create Pseudocode (Mar 11th - Mar 19th)</u>

- Start setting up and implementing symbols and opcode tables (hand-written).
- Implement command-line argument handling and error handling, e.g. terminate if no input files are given, display error message if input file is not supported.
- Design how pass 1 and pass 2 will be connected.
- Discuss workload distribution for coding in the next phase. Code will be individually committed to a github directory.
- Discuss any possible problems regarding functionality of the program and come up with alternatives before coding.
- Finalize design and get ready for Exam 2, Mar 19th

Estimated hours: 9

<u>Phase Three: Initial Stage Coding (Mar 20th - Mar 31st)</u>

- Start coding from design and pseudocode
- Implement a two-pass assembler that can assemble valid SIC/XE programs.
- Ensure object code is correctly generated for all addressing modes.
- Combine code created from both group members for any potential debugging and initial testing of implementation.
- Aim for a functional code to run without any error.
- Set goals to do during Spring Break, Mar 31st

Estimated hours: 20+

<u>Phase Four: Testing, Optimizing, and Debugging (Apr 1st- Apr 21)</u>

- Finalize coding by debugging any errors.
- Conduct multiple tests to ensure the program can run all of its functions smoothly, including validate SIC/XE instruction formats, choosing correct addressing modes and assembler directives.
- Note any errors or functions that are not up to standard and fix it
- Complete any documentation, README, comments on code.
- Final submission Apr 21th.

Estimated hours: 10-20

Functional and System Requirements:

Assembler Requirements:

- Accept input XE source files
- Output error messages if no input file is passed or wrong file format, and exit/terminate the program
- Successfully open the files
- Implement first pass
    - Assign addresses to instructions

    - Check if assembly directives and opcode is supported by checking OPTAB (manually created) and assembly directive file (manually created)
    - Define labels and its corresponding memory address
    - Create a symbol table
- Implement second pass
    - Use the SYMTAB and OPTAB to generate a listing file
    - Generate a symbol table file
- Process assembly directives on both passes
- Exit/terminate the program

Symbol Table:

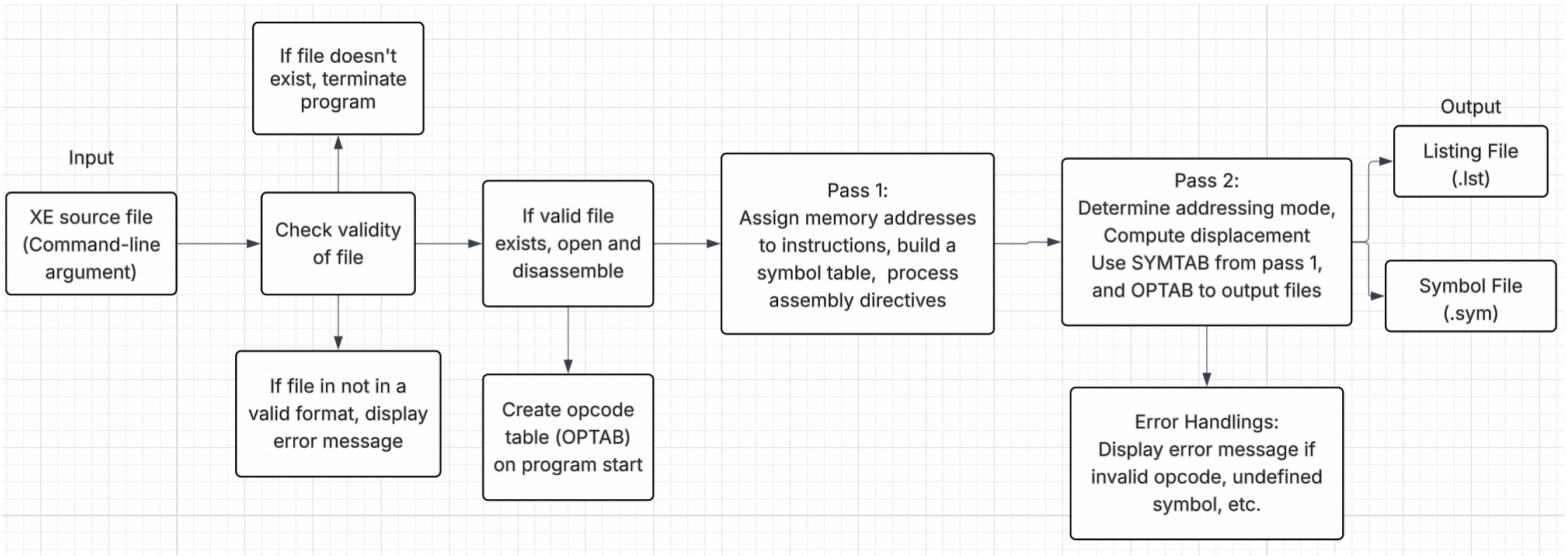
- Generated during the first pass, it contains the label name and its corresponding memory address

Opcode Table:

- Manually generated by the programmer beforehand
- Have mnemonics and its corresponding Hex value

System Design/Specification:

Data Flow Diagram



Our design follows a clear two-pass structure to ensure we can accurately read XE assembly source files and return a symbol table file and a listing file.

Opcode table will be hand-written, built and ran as program start

Pass 1 is responsible for assigning memory addresses to instructions, building a symbol table, and process assembly directives.

Pass 2 will first determine addressing mode to determine addressing type, calculate displacement and set nixbpe bits accompanied with SYMTAB and OPTAB to generate the final listing file and symbol file.

We do error handling throughout the whole process, detecting invalid file input and any invalid opcode or undefined symbols.

Development Environment:
Operating System: Windows OS (and Edoras for testing)
Integrated Developing Environment: Visual Studio Code
Language: C/C++
Compiler: GCC/G++ Compiler
Collaboration**:** GitHub
EDORAS from SDSU

Run/Test Environment:
Running environment:

- Use the Edoras OS server, G++ compiler environment to ensure it runs in that environment
- Compile the code folder with “make” command (without the double quotation)
- Execute the executable code with a command “./lxe” along with the testing input XE source files
- Check for any runtime errors, interrupt handlings, system calls issues if used in any part of the code

Code Review:

- Team members will check each other’s code and ensure that the code is concise, clear, well documented, and uses no magic numbers to avoid confusion
- Both team members will check for any syntax errors in the code and in the source files to ensure that the program will successfully run
- Both team members will check to ensure that all requirements are met

Testing:

- Both team members will create their own simple, testing input XE source files and test each pass (pass 1 and pass 2) individually to ensure that both logic functions are properly fulfilling its purpose
- Combine all of the testing units and develop a full XE source files to check if all parts of the program are working
- Check the output files and compare them with the expected outputs